Solving Recurrences — Substitution (Guess & Induction) Method

Detailed step-by-step solutions for three common recurrences (A, B, C). Each problem is solved by unrolling and then proven by induction.

Problem A Solve $T(n) = T(n/2) + C$

$$T(n) = T\left(\frac{n}{2}\right) + C$$

Unrolling / pattern discovery

Expand repeatedly (assume n is a power of 2 for clarity):

```
T(n) = T(n/2) + C
= T(n/4) + C + C = T(n/4) + 2C
= T(n/8) + 3C
...
After k expansions: T(n) = T(n/2^k) + k·C
```

Stop when $n/2^k = 1 \Rightarrow 2^k = n \Rightarrow k = \log_2 n$.

Closed form from unrolling:

$$T(n) = T(1) + C \log_2 n.$$

If $T(1) = a$, then $T(n) = a + C \log_2 n$.

Formal proof by induction (substitution method)

1. **Claim:** $T(n) \le a + C \log_2 n$ for all $n = 2^k$.
2. **Base** $n = 1$: $T(1) = a$ — true.
3. **Inductive step:** assume true for $n/2$:

$$T(n/2) = a + C \log_2(n/2).$$

Then

$$T(n) = T(n/2) + C = \big(a + C \log_2(n/2)\big) + C = a + C(\log_2 n - 1) + C = a + C \log_2 n.$$

Therefore $T(n) = \Theta(\log n)$.

Problem B Solve $T(n) = T(n-1) + C$

$$T(n) = T(n-1) + C$$

Unrolling / pattern discovery

Expand step-by-step:

```
T(n) = T(n-1) + C
= T(n-2) + 2C
= T(n-3) + 3C
...
After k steps: T(n) = T(n-k) + k·C
```

Stop at base $T(1)$ when $k = n - 1$. So

$$T(n) = T(1) + (n-1)C.$$

If $T(1) = a$, then $T(n) = a + (n-1)C$.

Formal proof by induction (substitution)

1. **Claim:** $T(n) = a + (n-1)C$ for all integers $n \geq 1$.
2. **Base** $n = 1$**:** $T(1) = a$ — true.
3. **Inductive step:** assume $T(n-1) = a + (n-2)C$. Then

$$T(n) = T(n-1) + C = \big(a + (n-2)C\big) + C = a + (n-1)C.$$

Therefore $T(n) = \Theta(n)$.

Problem C Solve $T(n) = 2T(n/2) + n$

$$T(n) = 2T\left(\frac{n}{2}\right) + n$$

Unrolling / pattern discovery

Expand once, twice, ... to find a pattern. Assume $n = 2^k$:

```
T(n) = 2 T(n/2) + n
= 2[2 T(n/4) + n/2] + n
= 4 T(n/4) + 2n
= 4[2 T(n/8) + n/4] + 2n
= 8 T(n/8) + 3n
...
After i expansions: T(n) = 2^i T(n/2^i) + i·n
Stop when n/2^i = 1 => i = log2 n
```

Substitute $i = \log_2 n$ and $T(1) = d$:

$$T(n) = 2^{\log_2 n} T(1) + n \log_2 n = n \cdot d + n \log_2 n.$$

Formal substitution (upper bound) — prove $T(n) = O(n \log n)$

1. Claim: There exist constants $A, B > 0$ such that for all sufficiently large n,

$$T(n) \leq An \log_2 n + Bn.$$

2. Base: Choose $B \geq T(1)$ so the base case holds for $n = 1$.
3. Inductive step: assume the inequality for smaller inputs:

$$T(n/2) \leq A\frac{n}{2}\log\frac{n}{2} + B\frac{n}{2}.$$

Then

$$T(n) = 2T(n/2) + n \leq 2\left(A\frac{n}{2}(\log n - 1) + B\frac{n}{2}\right) + n = An \log n + (B + 1 - A)n.$$

Choose $A \geq 1$ so that $(B + 1 - A) \leq B$. Thus the bound closes.

Formal substitution (lower bound) — prove $T(n) = \Omega(n \log n)$

1. Pick $a > 0$ (for example $a = 1/2$). Assume for smaller inputs, $T(m) \geq a\, m \log m$.
2. Then

$$T(n) = 2T(n/2) + n \geq 2\left(a\frac{n}{2}(\log n - 1)\right) + n = an \log n + (1 - a)n,$$

which for $a \leq 1$ gives a valid lower bound of order $n \log n$.

Combining both bounds yields:

$$T(n) = \Theta(n \log n).$$

(This result matches the recursion-tree and Master Theorem.)